Entrega aquest mateix document (format word o PDF) amb les respostes (preferiblement usant un altre color, com per exemple blau).

Exercicis:

1. **En una biblioteca digital es programa una classe Llibre. El professor diu:**

   "He creat Llibre i ara tinc 3 llibres al meu programa."

   Indica si les frases són correctes i justifica breument:

   a) "Llibre és un objecte."
   No. Llibre és la classe. Un "objecte" seria un llibre concret creat a partir d'aquesta classe. La classe defineix com seran els objectes, però no és l'objecte en si.

   b) "Els 3 llibres són instàncies de Llibre."
   Si. En POO, quan creem un objecte concret a partir d'una classe, diem que l'estem "instanciant".
   c) "Tots els llibres comparteixen el mateix titol perquè venen de la mateixa classe."
   No. Tots els llibres comparteixen l'atribut "títol", però cada instància té el seu propi valor (el seu propi estat).
   d) "Puc crear més llibres amb new Llibre(...)."
   Si. La paraula clau new es com es fan mes llibres.

2. **Per a cada frase, identifica:**

   - Quina és la classe
   - Quines són les propietats de la classe
   - Quins són els mètodes de la classe
   - Quin és l'objecte de la classe, és a dir, la instància de la classe

   a) "La porta de l'aula 2 està tancada i feta de fusta; quan arriba el conserge, l'obre."
   Clase: Porta
   Propietats: Tancada, fusta
   Mètodes: Obrirla, tancarla
   Objecte: La porta de l'aula 2
   b) "La comanda #A104 té 3 productes i està en estat 'pendent'; el sistema calcula el total abans de pagar-la."
   Clase: Comanda
   Propietats: Pendent, pagada
   Mètodes: Calcula el total
   Objecte: La comanda #A104
   c) "La playlist 'Gym' té 12 cançons; en prémer un botó, s'afegeix una cançó nova."
   Clase: playlist
   Propietats: cançons
   Mètodes: Afegir canço

Objecte: Playlist ‘Gym’

d) “La reserva R-77 és per a divendres i per a 4 persones; el recepcionista la cancel·la.”

Clase: Reserva

Propietats: data limit, persones, cançelada

Mètodes: Cançelar

Objecte: La reserva R-77

e) “El termòmetre del laboratori marca 18°C; en engegar-se el sistema, s’actualitza la lectura.”

Clase: termòmetre

Propietats: graus

Mètodes: Engegar, actualitzar lectura

Objecte: El termòmetre del laboratori

3. **Tenim la següent regla:**

“Una CompteBancari mai no pot tenir el saldo negatiu”.

Els objectius que tenim marcats són:

- En crear el compte el saldo val 0.
- Hem de permetre ingressar i retirar saldo.

Hem plantejat dos dissenys de la classe CompteBancari, però no sabem quin dels dos és millor. Hem fet dues proves al **main** (una amb cada disseny de la classe):

A)

```
CompteBancari c = new CompteBancari("Anna");
c.saldo = 50;
c.saldo = c.saldo - 100;
```

B)

```
CompteBancari c = new CompteBancari("Anna");
c.ingressar(50);
c.retirar(100);
```

1. Sense veure el disseny de cada classe, tria quin dels dos dissenys és millor.
   El segon.

2. Justifica amb un exemple de “què podria anar malament” a l’opció dolenta.
   El problema de l'opció A és que trenca l'encapsulament. En fer l'atribut `saldo` públic (accessible directament des de fora), la classe perd el control sobre el seu propi estat. No pot validar res.
   Exemple:
   // Opció A (Dolenta)
   c.saldo = 50;

```
// L'usuari intenta treure 100. El programa fa la resta: 50 - 100 = -50.
// Com que és una assignació directa, la classe NO pot dir "no".
c.saldo = c.saldo - 100;
System.out.println(c.saldo); // Resultat: -50
```

3. Posa un exemple de com l'opció bona podria gestionar aquesta situació (expliqueu o posa codi directament).

```
public class CompteBancari {
    // 1. L'atribut és privat: NINGÚ el toca directament des de fora
    private double saldo;

    public CompteBancari(String titular) {
        this.saldo = 0; // Inicialitzem a 0 com demanava l'enunciat
    }

    // 2. Mètode per retirar diners amb CONTROL (validació)
    public void retirar(double quantitat) {
        // Aquesta és la màgia: comprovem la regla abans de restar
        if (quantitat > this.saldo) {
            System.out.println("Error: Fons insuficients. Operació cancel·lada.");
        } else {
            this.saldo = this.saldo - quantitat;
            System.out.println("Retirada acceptada. Nou saldo: " + this.saldo);
        }
}

    // Mètode per ingressar (aquí no cal tanta validació, però és bona pràctica)
    public void ingressar(double quantitat) {
        if (quantitat > 0) {
            this.saldo = this.saldo + quantitat;
        }
    }
}
```

4. **Tenim la següent classe:**

```
class Termostat {
        public int temperatura;
        public boolean encès;
}
```

Al main algú fa:

```
Termostat t = new Termostat();
t.temperatura = -200;
t.encès = true;
```

1. Assenyala 2 problemes de disseny.
   Accedir a la variable temperatura directament amb la variable i no amb un setter i posar -200 que és imposible

2. Proposa millores concretes i justifica les teves eleccions.
   Per millorar els anteriors problemes posaria un setter de la temperatura a la clase Termostat amb un filtre de temperatura mínima i màxima

5. En un Termostat hi ha l'atribut temperaturaActual i l'atribut encès. Normes:

- Si està apagat, no es pot canviar la temperatura.
- La temperatura vàlida està entre 5 i 30.

Completa els buits i respon a la pregunta:

```
public void ajustar(int novaTemp) {

        if (!ences) {
                System.out.println("No es pot ajustar: apagat");
                return;
        }

        if (novaTemp < 5) {
                novaTemp = 5;
        }

        if (novaTemp > 30) novaTemp = 30;
                this.temperaturaActual= novaTemp;
        }
}
```

Per què no és bona idea permetre que el main escrigui directament temperaturaActual sense passar per ajustar?

Perquè podria ser que el termostat estigui apagat o que s'intenti posar una temperatura estupida

6. Es vol una classe Producte amb:

- nom
- preu (no pot ser negatiu; si ho és, queda a 0)

Constructor proposat:

```
public Producte(String nom, double preu) {
        this.nom = nom;
```

```
        this.preu = preu;
    }
```

Respon les següents preguntes:

1. Explica l'error (si n'hi ha).
   No hi ha filtre del preu
2. Escriu una línia que el corregeixi (només la part del preu).
   if(preu<0){
   this.preu=preu;
   }

7. **Suposa una classe CompteBancari ben dissenyada (saldo no negatiu). Codi del main:**

```
CompteBancari c = new CompteBancari("Anna", 20);
c.retirar(5);
c.retirar(30);
c.ingressar(10);
c.imprimirResum();
```

Respon les següents preguntes:

1. Quin hauria de ser el saldo final?
   10

2. Què hauria de passar a retirar(30)? Per què?
   El saldo s'ha de posar a 0 ja que no hi ha números negatius (20-5=15, 15-30=-15=0)

8. **Per a cada cas, digues si faries servir herència o no (justifica les respostes):**

a) Gat i Animal:
Si perquè el gat te propietats d'animal
b) Comanda i Client:
No perquè son coses distintes
c) Empleat i Persona:
Si perquè l'empleat es una persona
d) Cuina i Casa:
No perquè una cuina te diferents funcions que una casa

9. **Completa el codi i respon la pregunta:**

```
class Notificacio {
        protected String missatge;
```

```
        public Notificacio(String missatge) {
                this.missatge = missatge;
        }

        public void enviar() {
                System.out.println("Enviant: " + missatge);
        }
}

class NotificacioEmail extends Notificacio {

        public NotificacioEmail(String missatge) {
                super(missatge);
        }

        public void enviar() {
                System.out.println("EMAIL -> " + missatge);
        }
}
Ara:
Notificacio n = new NotificacioEmail("Hola");
n.enviar();
```

Què imprimeix? Per què?

Imprimira el 'EMAIL → Hola perquè el new es de la clase NotificacioEmail

10. Fes un programa en Java per gestionar "tasques". Hi ha tasques normals i tasques amb data límit.

1. Has de crear:
   - 1 classe pare (tasca genèrica)
   - 2 classes filles (dos tipus de tasca diferents)
2. Totes les tasques han de tenir:
   - un títol
   - una descripció
3. Cada tipus concret de tasca ha d'afegir una dada pròpia (tria tu quina).
4. Totes les tasques han de tenir dos mètodes:
   a. Un que mostri la informació que tenen en comú (és a dir, que tenen tots els tipus de tasques)
   b. Un que mostri la informació pròpia.
5. Main (obligatori):

- Crea 3 objectes (1 pare + 2 filles).
- Desa'ls en un array o llista del tipus del pare.
- Recorre'ls i mostra la informació comuna i la pròpia (dos mètodes).

Entrega (NO entregar codi ni arxius .java):

- **Captures del codi (3 classes + main)**
- **Captura de la sortida de consola**

Pots aferrar aquí les captures de pantalla:

```java
//TIP To <b>Run</b> code, press <shortcut actionId="Run"/> or
// click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.
void main() {
    Tasca tasca = new Tasca("Matematicas","Deberes de matematicas");
    TascaPrioritat tascaPrioritat = new TascaPrioritat("Alta","programación","Deberes de programación");
    TascaDataLimit tascaDataLimit = new TascaDataLimit("30/01/206","lenguaje de Marcas","Deberes de lenguaje de Marcas");
    List<Tasca> tasques = new ArrayList<>();
    tasques.add(tasca);
    tasques.add(tascaPrioritat);
    tasques.add(tascaDataLimit);
    for(Tasca t : tasques){
        System.out.println("-------");
        t.informacio();
        t.particular();

    }
}
```

```java
public class Tasca {
    protected String titol;
    protected String descripcio;

    public Tasca(String titol,String descripcio){
        this.titol=titol;
        this.descripcio=descripcio;

    }

    public void informacio(){
        System.out.println(titol+" "+descripcio);
    }

    void particular(){
        System.out.println("No té particularitat");
    }

}
```

```java
public class TascaDataLimit extends Tasca {
    private String dataLimit;
    public TascaDataLimit(String dataLimit, String titol, String descripcio){
        super(titol,descripcio);
        this.dataLimit=dataLimit;
    }
    @Override
    public void informacio(){
        System.out.println(titol+" "+descripcio);
    }

    @Override
    public void particular(){
        System.out.println(dataLimit);
    }
}
```

```java
public class TascaPrioritat extends Tasca {
    private String prioritat;

    public TascaPrioritat(String prioritat, String titol, String descripcio){
        super(titol,descripcio);
        this.prioritat=prioritat;
    }
    @Override
    public void informacio(){
        System.out.println(titol+" "+descripcio);
    }
    @Override
    public void particular(){
        System.out.println(prioritat);
    }
}
```

```
"C:\Program Files\Eclipse Adoptium\jdk-25.0.1.8-hotspot\bi
-------
Matematicas Deberes de matematicas
No té particularitat
-------
programación Deberes de programación
Alta
-------
lenguaje de Marcas Deberes de lenguaje de Marcas
30/01/206

Process finished with exit code 0
```